SCR2043 OPERATING SYSTEMS

Name	:	Goe Jie Ying	Marks
Student ID	:	A23CS0224	
Section	:	3	

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. **Download necessary source codes:**

   Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. **Compile the source files:**

   Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. **Execute the dummy processes:**

   Run all the dummy processes

```
./mainprocess &
```

   Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command
`ps -ef`
Output

```
jieying      937       1  0 09:23 ?        00:00:00 /usr/lib/systemd/systemd --user
jieying      938     937  0 09:23 ?        00:00:00 (sd-pam)
jieying      951     780  0 09:23 tty1     00:00:00 -bash
jieying      983     951  0 09:26 tty1     00:00:00 ./mainprocess
jieying      984     983  0 09:26 tty1     00:00:00 ./mainprocess
jieying      985     983  0 09:26 tty1     00:00:00 ./mainprocess
jieying      986     984  0 09:26 tty1     00:00:00 ./subprocess1
jieying      987     984  0 09:26 tty1     00:00:00 ./subprocess1
jieying      988     985  0 09:26 tty1     00:00:00 ./subprocess2
jieying      989     985  0 09:26 tty1     00:00:00 ./subprocess2
jieying      990     985  0 09:26 tty1     00:00:00 ./subprocess2
root         997       2  0 09:31 ?        00:00:00 [kworker/u6:0-events_unbound]
root        1001       2  0 09:34 ?        00:00:00 [kworker/u5:1-events_power_efficient]
root        1005       2  0 09:36 ?        00:00:00 [kworker/u6:3-events_unbound]
root        1009       2  0 09:37 ?        00:00:00 [kworker/1:0-cgroup_destroy]
jieying     1016     951  0 09:41 tty1     00:00:00 ps -ef
```

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command
`ps -ef |grep -E 'mainprocess|subprocess'`
Output

```
jieying@secr2043:~$ ps -ef| grep -E 'subprocess'
jieying      986     984  0 09:26 tty1     00:00:00 ./subprocess1
jieying      987     984  0 09:26 tty1     00:00:00 ./subprocess1
jieying      988     985  0 09:26 tty1     00:00:00 ./subprocess2
jieying      989     985  0 09:26 tty1     00:00:00 ./subprocess2
jieying      990     985  0 09:26 tty1     00:00:00 ./subprocess2
jieying     1036     951  0 09:55 tty1     00:00:00 grep --color=auto -E subprocess
jieying@secr2043:~$ ps -ef |grep -E 'mainprocess|subprocess'
jieying      983     951  0 09:26 tty1     00:00:00 ./mainprocess
jieying      984     983  0 09:26 tty1     00:00:00 ./mainprocess
jieying      985     983  0 09:26 tty1     00:00:00 ./mainprocess
jieying      986     984  0 09:26 tty1     00:00:00 ./subprocess1
jieying      987     984  0 09:26 tty1     00:00:00 ./subprocess1
jieying      988     985  0 09:26 tty1     00:00:00 ./subprocess2
jieying      989     985  0 09:26 tty1     00:00:00 ./subprocess2
jieying      990     985  0 09:26 tty1     00:00:00 ./subprocess2
jieying     1040     951  0 09:57 tty1     00:00:00 grep --color=auto -E mainprocess|subprocess
```

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command
`ps -ef| grep -E 'subprocess'`
Output

```
jieying@secr2043:~$ ps -ef| grep -E 'subprocess'
jieying      986     984  0 09:26 tty1     00:00:00 ./subprocess1
jieying      987     984  0 09:26 tty1     00:00:00 ./subprocess1
jieying      988     985  0 09:26 tty1     00:00:00 ./subprocess2
jieying      989     985  0 09:26 tty1     00:00:00 ./subprocess2
jieying      990     985  0 09:26 tty1     00:00:00 ./subprocess2
jieying     1036     951  0 09:55 tty1     00:00:00 grep --color=auto -E subprocess
```

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command
`ps -eo pid,user,pcpu,pmem,cmd | grep -E 'subprocess'`
Output

```
jieying@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd|grep -E 'subprocess'
    973 jieying   0.0  0.1 ./subprocess2
    974 jieying   0.0  0.1 ./subprocess1
    975 jieying   0.0  0.1 ./subprocess2
    976 jieying   0.0  0.1 ./subprocess1
    977 jieying   0.0  0.1 ./subprocess2
    979 jieying   0.0  0.2 grep --color=auto -E subprocess
```

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
`ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem | grep -E 'subprocess'`
Output

```
jieying@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd --sort=-pmem | grep -E 'subprocess'
    995 jieying   0.0  0.2 grep --color=auto -E subprocess
    973 jieying   0.0  0.1 ./subprocess2
    974 jieying   0.0  0.1 ./subprocess1
    975 jieying   0.0  0.1 ./subprocess2
    976 jieying   0.0  0.1 ./subprocess1
    977 jieying   0.0  0.1 ./subprocess2
```

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
`ps -ef --forest -C mainprocess -C subprocess1 -C subprocess2`
Output

```
jieying      961     779  0 10:15 tty1     00:00:00  \_ -bash
jieying      970     961  0 10:16 tty1     00:00:00      \_ ./mainprocess
jieying      971     970  0 10:16 tty1     00:00:00      |   \_ ./mainprocess
jieying      974     971  0 10:16 tty1     00:00:00      |   |   \_ ./subprocess1
jieying      976     971  0 10:16 tty1     00:00:00      |   |   \_ ./subprocess1
jieying      972     970  0 10:16 tty1     00:00:00      |   \_ ./mainprocess
jieying      973     972  0 10:16 tty1     00:00:00      |       \_ ./subprocess2
jieying      975     972  0 10:16 tty1     00:00:00      |       \_ ./subprocess2
jieying      977     972  0 10:16 tty1     00:00:00      |       \_ ./subprocess2
jieying     1028     961  0 10:34 tty1     00:00:00      \_ ps -ef --forest -C mainprocess -C subprocess1 -C subprocess2
jieying      946       1  0 10:15 ?        00:00:00 /usr/lib/systemd/systemd --user
jieying      947     946  0 10:15 ?        00:00:00  \_ (sd-pam)
```

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command
`pstree -c -s $$`
Output

```
jieying@secr2043:~$ pstree -c -s $$
systemd──login──bash─┬─mainprocess─┬─mainprocess─┬─subprocess1
                     │             │             └─subprocess1
                     │             └─mainprocess─┬─subprocess2
                     │                           ├─subprocess2
                     │                           └─subprocess2
                     └─pstree
```

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
`sudo renice -n 5 -p 974`
Output

```
jieying@secr2043:~$ ps -C subprocess1
    PID TTY          TIME CMD
    974 tty1     00:00:00 subprocess1
    976 tty1     00:00:00 subprocess1
jieying@secr2043:~$ sudo renice -n 5 -p 976
976 (process ID) old priority 0, new priority 5
jieying@secr2043:~$ sudo renice -n 5 -p 974
974 (process ID) old priority 0, new priority 5
```

Question 9

Terminate all running processes with the name “mainprocess”.

Command
`killall -15 mainprocess`
Output

```
jieying@secr2043:~$ killall -15 mainprocess
Main process (ID: 970) received signal: 15. Terminating...
Main process (ID: 971) received signal: 15. Terminating...
[1]+  Done                    ./mainprocess
jieying@secr2043:~$ Main process (ID: 972) received signal: 15. Terminating...
```

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(){
    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    } else if(pid ==0) {
        printf("Child process: PID = %d, Parent PID = %d\\n", getpid(), getppid());
    }else {
        wait(NULL);
        printf("Parent process: PID = %d, Child PID = %d finished\n", getpid(), pid);
    }

    return 0;
}
```

Output:

Nao

```
jieying@secr2043:~$ gcc fork_wait.c -o fork_wait
jieying@secr2043:~$ ./fork_wait
Child process: PID = 996, Parent PID = 995\nParent process: PID = 995, Child PID = 996 finished
```